```
@Configuration
public class RateLimiterConfig {

  @Bean
  KeyResolver keyResolver() {
    return exchange -> exchange.getPrincipal()     ❶
      .map(Principal::getName)                     ❷
      .defaultIfEmpty("anonymous");                ❸
  }
}
```

❶ Gets the currently authenticated user (the principal) from the current request (the exchange)

❷ Extracts the username from the principal

❸ If the request is unauthenticated, it uses “anonymous” as the default key to apply rate-limiting.

That concludes the basic configuration for authenticating users of Polar Bookshop using OpenID Connect. The following section will cover how logout works in Spring Security and how we can customize it for the OAuth2/OIDC scenario.

11.4.5 Configuring user logout in Spring Security and Keycloak

So far, we have addressed the challenges and solutions for authenticating users in a distributed system. Still, we should consider what happens when a user logs out.

In Spring Security, logging out results in all the session data associated with the user being deleted. When OpenID Connect/OAuth2 is used, the tokens stored by Spring Security for that user are also deleted. However, the user will still have an active session in Keycloak. Just as the authentication process involves both Keycloak and Edge Service, completely logging a user out requires propagating the logout request to both components.